

Metody Obliczeniowe w Nauce i Technice

Laboratorium 7 – Kwadratury, część II

Mateusz Stoch

28 kwietnia 2026

1 Zadanie 1 – Prędkość strzały z łuku

1.1 Opis zadania

Dane są wartości siły naciągu F łuku w funkcji długości naciągu x :

x [m]	0.00	0.05	0.10	0.15	0.20	0.25	0.30	0.35	0.40	0.45	0.50
F [N]	0	37	71	104	134	161	185	207	225	239	250

Łuk jest naciągnięty na odległość $L = 0,5$ m, a masa strzały wynosi $m = 0,075$ kg. Energia kinetyczna strzały w chwili wypuszczenia równa się pracy włożonej w naciągnięcie łuku:

$$\frac{1}{2}mv^2 = \int_0^L F dx$$

1.2 Argumentacja rozwiązania

Dane są podane w postaci dyskretnych punktów pomiarowych, a nie analitycznej funkcji, dlatego całkę wyznaczono metodą Simpsona z biblioteki SciPy (`scipy.integrate.simpson`). Metoda Simpsona przy równoodległych węzłach jest odpowiednim wyborem, ponieważ minimalizuje błąd przy stosunkowo niewielu punktach pomiarowych.

```
1 x_data = np.array([0.00, 0.05, 0.10, ..., 0.50])
2 F_data = np.array([0, 37, 71, ..., 250])
3
4 W_simpson = np.float64(integrate.simpson(F_data, x=x_data))
5 v_simpson = np.sqrt(np.float64(2.0) * W_simpson / m)
```

Listing 1: Obliczenie pracy i prędkości strzały

1.3 Wyniki



Wykres 1: Siła naciagu łuku w zależności od długości naciagu. Zacieniowany obszar odpowiada pracy W .

Obliczone wartości:

- Praca: $W = 74,5333 \text{ J}$
- Prędkość strzały: $v = 44,5820 \text{ m/s} \approx 160 \text{ km/h}$

1.4 Wnioski

Metoda Simpsona dobrze nadaje się do całkowania danych dyskretnych.

2 Zadanie 2 – Ciało na sprężynie z tarciem

2.1 Część (a): Prędkość v_0 w punkcie $x = 0$

2.1.1 Opis zadania

Ciało o masie $m = 0,8 \text{ kg}$ jest przymocowane do sprężyny o stałej sprężystości $k = 80 \text{ N/m}$ i długości swobodnej $l = 0,4 \text{ m}$. Współczynnik tarcia między ciałem a poziomym prętem wynosi $\mu = 0,3$. Przyspieszenie ciała jest równe $\ddot{x} = -f(x)$, gdzie:

$$f(x) = \mu g + \frac{k}{m} (\mu l + x) \left(1 - \frac{l}{\sqrt{l^2 + x^2}} \right)$$

Prędkość ciała w punkcie $x = 0$, gdy zaczyna ono ruch od $x = l$, wyraża się wzorem:

$$v_0 = \sqrt{2 \int_0^l f(x) dx}$$

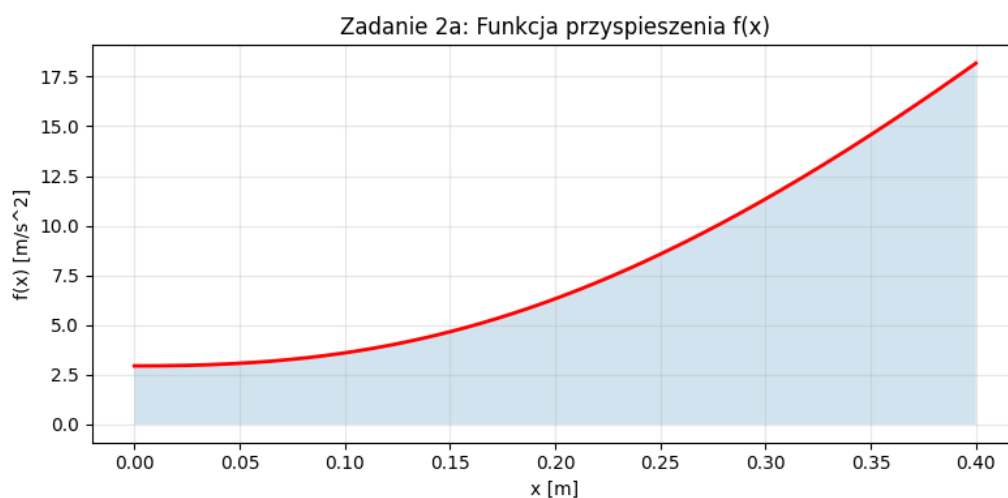
2.1.2 Argumentacja rozwiązania

Funkcja podcałkowa $f(x)$ jest gładka na przedziale $[0, l]$, więc można bezpośrednio zastosować adaptacyjną kwadraturę Gaussa-Kronroda (`scipy.integrate.quad`).

```
1 def f2(x):  
2     return mu*g + (k/m2)*(mu*l2 + x)*(1.0 - l2/np.sqrt(l2**2 + x**2))  
3  
4 W2, err2 = integrate.quad(f2, 0.0, float(l2))  
5 v0 = np.sqrt(2.0 * W2)
```

Listing 2: Obliczenie prędkości v_0

2.1.3 Wyniki



Wykres 2: Funkcja przyspieszenia $f(x)$ na przedziale $[0, l]$.

- Całka: $\int_0^l f(x) dx = 3,119190$
- Szacowany błąd: $3,46 \times 10^{-14}$
- Prędkość: $v_0 = 2,497675 \text{ m/s}$

2.2 Część (b): Stała C i czas t

2.2.1 Opis zadania

Czas potrzebny na przebycie drogi od $x = l$ do $x = 0$ wynosi $t = C\sqrt{m/k}$, gdzie:

$$C = \int_0^1 \left[(\sqrt{2} - 1)^2 - (\sqrt{1+z} - 1)^2 \right]^{-1/2} dz$$

Funkcja podcałkowa ma osobliwość w punkcie $z = 1$, która zachowuje się jak $(1 - z)^{-1/2}$.

2.2.2 Argumentacja rozwiązania

Aby poradzić sobie z osobliwością, zastosowano następujące podejście:

1. Bezpośrednie wywołanie `integrate.quad` z parametrem `points=[0.99]` – wymuszenie podziału siatki w pobliżu osobliwości.

```
1 def integrand_C(z):
2     A = (np.sqrt(2.0) - 1.0)**2
3     B = (np.sqrt(1.0 + z) - 1.0)**2
4     val = np.where(A - B <= 0.0, 1e-30, A - B)
5     return 1.0 / np.sqrt(val)
6
7 C_val, err_C = integrate.quad(
8     integrand_C, 0.0, 1.0,
9     limit=200, points=[0.99],
10    epsabs=1e-10, epsrel=1e-10
11 )
12 t_val = C_val * np.sqrt(m2 / k)
```

Listing 3: Obliczenie stałej C z obsługą osobliwości

2.2.3 Wyniki

- Stała $C = 3,970020$
- Szacowany błąd: $1,24 \times 10^{-10}$
- Czas: $t = C\sqrt{m/k} = 3,970020 \cdot 0,100000 = 0,397002\text{ s}$

2.3 Wnioski

Zadanie 2 pokazuje dwa typowe problemy praktyczne przy numerycznym całkowaniu:

- W części (a) funkcja jest gładka, co sprawia, że adaptacyjna kwadratura Gaussa-Kronroda daje wynik niemal natychmiastowo z precyzją rzędu błędu maszynowego.
- W części (b) istnieje osobliwość w końcowym punkcie przedziału, która wymaga specjalnego traktowania. Poinformowanie algorytmu o lokalizacji osobliwości (parametr `points`).

3 Zadanie 3 – Wahadło matematyczne

3.1 Opis zadania

Dane jest wahadło matematyczne o długości $l = 1\text{ m}$ i wychyleniu początkowym θ_0 . Dla małych wychyleń okres jest przybliżany wzorem:

$$T_0 = 2\pi\sqrt{\frac{l}{g}}$$

Dla dowolnych wychyleń dokładny okres wyraża się przez całkę eliptyczną pierwszego rodzaju $K(k)$:

$$T(\theta_0) = 4\sqrt{\frac{l}{g}} K(k), \quad K(k) = \int_0^{\pi/2} \frac{d\theta}{\sqrt{1 - k^2 \sin^2 \theta}}, \quad k = \sin \frac{\theta_0}{2}$$

3.2 Część (a): Całka eliptyczna kwadraturą

3.2.1 Argumentacja rozwiązania

Całkę eliptyczną wyznaczono przy użyciu `scipy.integrate.quad` z funkcją podcałkową $\frac{1}{\sqrt{1-k^2 \sin^2 \theta}}$ na przedziale $[0, \pi/2]$. Jako wartość referencyjną przyjęto wartość zwracaną przez funkcję `scipy.special.ellipk`.

```

1 k_mod = np.sin(theta0 / 2.0)
2 K_true = np.float64(special.ellipk(k_mod**2))
3
4 def elliptic_integrand(theta, k):
5     return 1.0 / np.sqrt(1.0 - k**2 * np.sin(theta)**2)
6
7 K_quad, err_K = integrate.quad(
8     elliptic_integrand, 0.0, np.pi/2, args=(float(k_mod),)
9 )
10 T_true_quad = 4.0 * np.sqrt(l3 / g3) * K_quad

```

Listing 4: Obliczenie całki eliptycznej kwadraturą

3.2.2 Wyniki

Dla $\theta_0 = 45^\circ$:

- Moduł: $k = \sin(22,5^\circ) \approx 0,38268343$
- $K(k)$ z `scipy.special.ellipk`: 1,63358631
- $K(k)$ kwadratura: 1,63358631
- Błąd względny (kwadratura vs scipy): $1,36 \times 10^{-16}$
- Dokładny okres: $T(\theta_0 = 45^\circ) = 2,086256 \text{ s}$
- Przybliżony okres (małe drgania): $T_0 = 2,006067 \text{ s}$

3.3 Część (b): Całka eliptyczna przez AGM

3.3.1 Opis zadania

Istnieje wzór na całkę eliptyczną przez średnią arytmetyczno-geometryczną (AGM):

$$K(k) = \frac{\pi}{2} \cdot \frac{1}{\text{AGM}(1+k, 1-k)}$$

Ciągi a_n, b_n definiujące AGM spełniają rekurencję:

$$a_0 = 1+k, \quad b_0 = 1-k, \quad a_{n+1} = \frac{a_n + b_n}{2}, \quad b_{n+1} = \sqrt{a_n b_n}$$

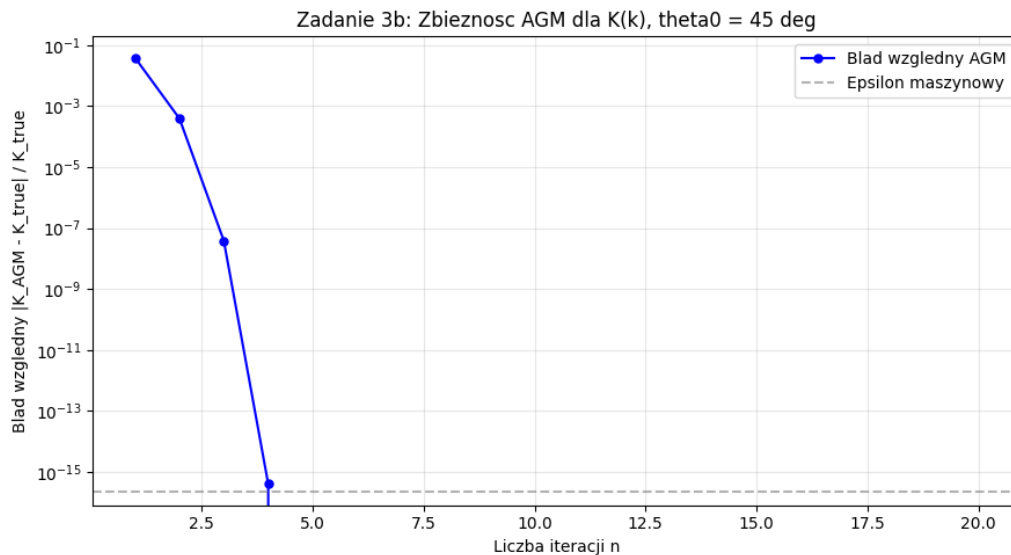
3.3.2 Argumentacja rozwiązania

AGM zbiega kwadratowo, więc kilka iteracji wystarczy, by osiągnąć precyzję maszynową. Zbieżność jest jeszcze szybsza niż wykładnicza.

```
1 def agm_K(k_val, n_iter):
2     a = 1.0 + k_val
3     b = 1.0 - k_val
4     for _ in range(n_iter):
5         a_new = (a + b) / 2.0
6         b_new = np.sqrt(a * b)
7         a, b = a_new, b_new
8     return (np.pi / 2.0) / a
```

Listing 5: Obliczenie $K(k)$ przez AGM

3.3.3 Wyniki



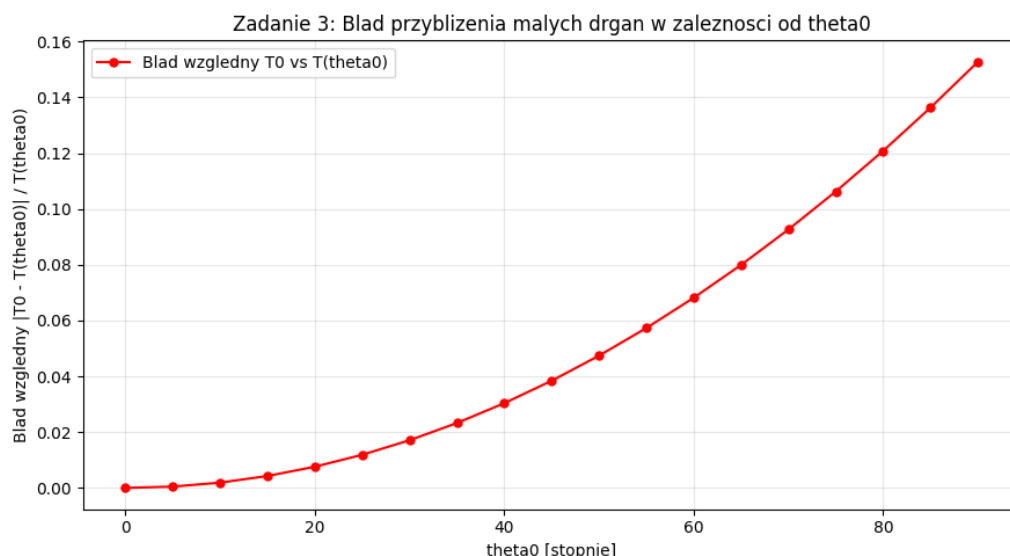
Wykres 3: Błąd względny przybliżenia $K(k)$ przez AGM w zależności od liczby iteracji ($\theta_0 = 45^\circ$).

Po 20 iteracjach:

- $K(k)_{\text{AGM}} = 1,63358631$
- Błąd względny AGM: $0,00 \times 10^0$ (poniżej progu precyzji maszynowej)
- Okres AGM: $T = 2,086256 \text{ s}$

Z wykresu wynika, że AGM osiąga precyzję maszynową już po około 4–5 iteracjach.

3.4 Błąd przybliżenia małych drgań



Wykres 4: Błąd względný okresu T_0 (przybliżenie małych drgań) w zależności od amplitudy θ_0 .

Wykres pokazuje, że błąd wzoru $T_0 = 2\pi\sqrt{l/g}$ rośnie gwałtownie wraz ze wzrostem kąta wychylenia. Dla małych kątów (do ok. 15°) błąd jest pomijalny, natomiast dla $\theta_0 = 90^\circ$ przekracza już kilkanaście procent.

3.5 Wnioski

1. Kwadratura adaptacyjna Gaussa-Kronroda (`quad`) wyznacza całkę eliptyczną z precyzją bliską granicy maszynowej.
2. Metoda AGM zbiega znacznie szybciej niż typowe kwadratury numeryczne, ponieważ jej zbieżność jest kwadratowa. W praktyce 5 iteracji wystarczy do osiągnięcia pełnej precyzji `float64`.
3. Przybliżenie okresu małych drgań T_0 jest wystarczająco dobre tylko dla kątów wychylenia nieprzekraczających kilkunastu stopni. Przy większych kątach należy korzystać ze wzoru dokładnego opartego na całce eliptycznej.

4 Podsumowanie

Laboratorium 7 skupiło się na zastosowaniach całkowania numerycznego do problemów fizycznych. Wykazano, że:

- Do całkowania danych tabelarycznych (zadanie 1) wygodnie stosuje się prostą kwadraturę Simpsona.
- Adaptacyjna kwadratura Gaussa-Kronroda (`scipy.integrate.quad`) radzi sobie znakomicie z gładkimi funkcjami i funkcjami z osobliwościami po wskazaniu ich lokalizacji (zadanie 2).

- Dla specjalnych funkcji matematycznych (całka eliptyczna) istnieją efektywne algorytmy iteracyjne (AGM), które osiągają bardzo wysoką precyzję przy minimalnej liczbie obliczeń (zadanie 3).